Union Find

AlgoExpert · Medium · Famous Algorithms

The problem

- Disjoint-set with createSet, find (null if unknown) and union, using path compression and union by rank.

Intuition

- Each set is a tree with a representative root; path compression and union by rank keep trees flat.

Brute force vs optimal

- Naive: label every element with its set ID → union is O(n)
- Plain parent trees → find can be O(n)
- Rank + path compression → nearly O(1) amortised (α(n))

Approach

- 1. createSet makes a single-node tree
- 2. find follows parents with path compression
- 3. union attaches the shorter tree under the taller

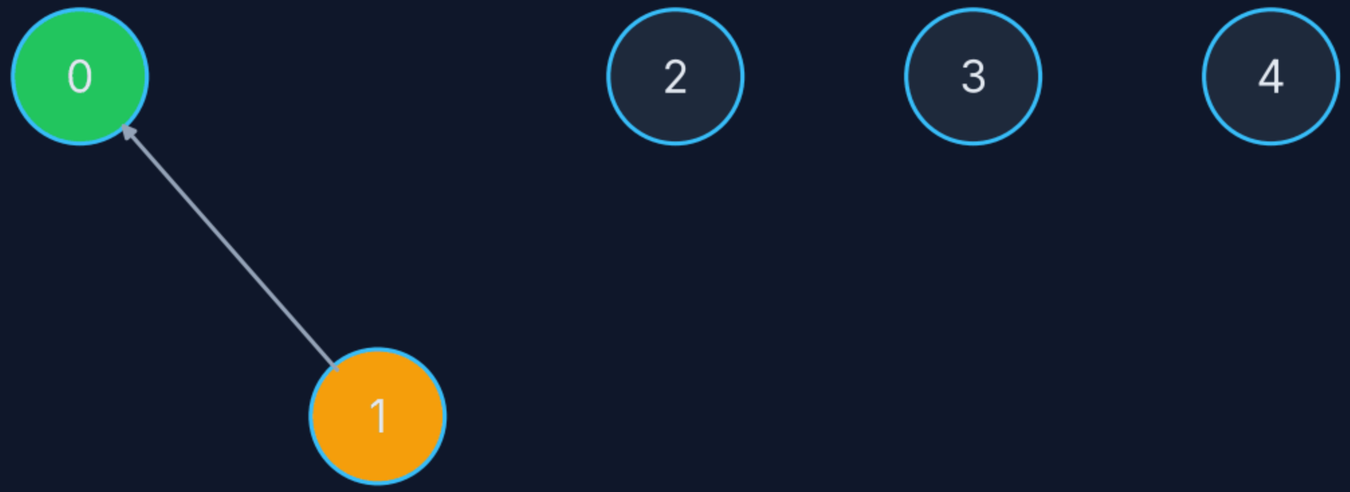
Worked example




```
parent: [0, 1, 2, 3, 4]
rank:   [0, 0, 0, 0, 0]
```

createSet(0...4): every node is its own root

Worked example

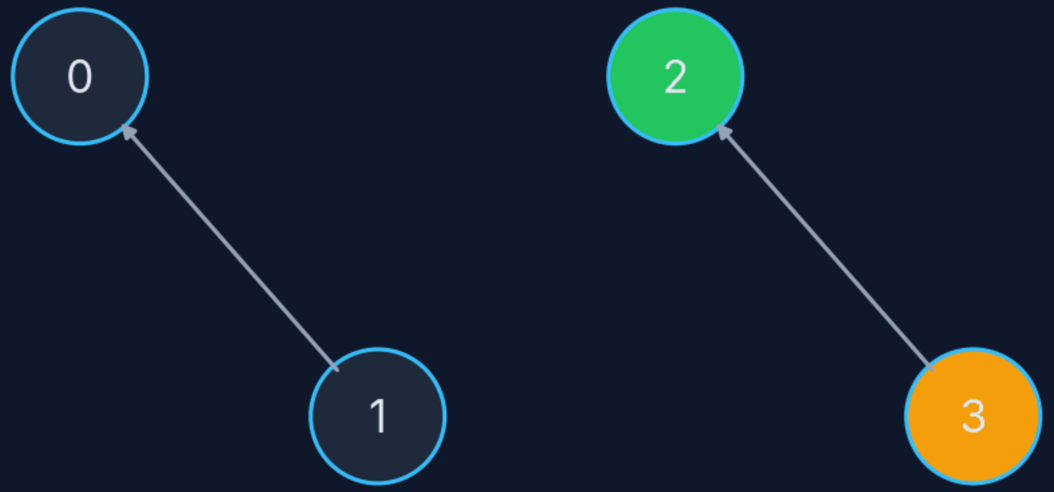



```
parent: [0, 0, 2, 3, 4]

rank:   [1, 0, 0, 0, 0]
```

union(0, 1): roots 0, 1 → attach by rank

Worked example

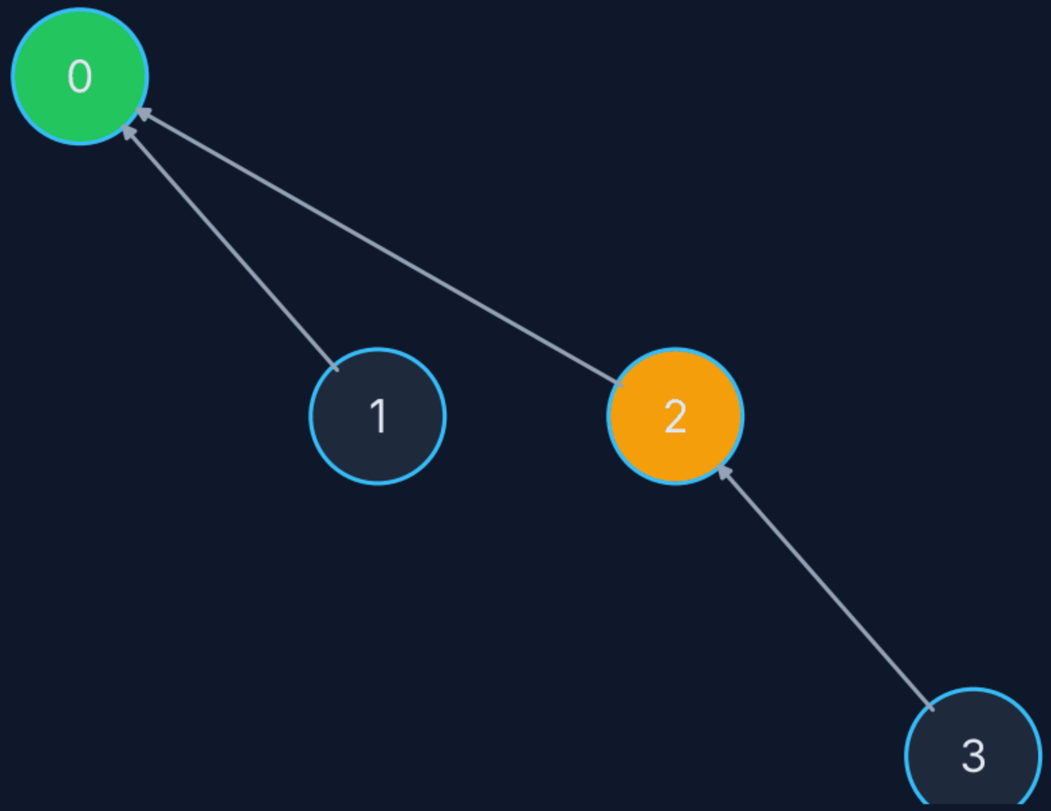



```
parent: [0, 0, 2, 2, 4]

rank:   [1, 0, 1, 0, 0]
```

union(2, 3): roots 2, 3 → attach by rank

Worked example




```
parent: [0, 0, 0, 2, 4]

rank:   [2, 0, 1, 0, 0]
```

union(0, 3): roots 0, 2 → attach by rank

Worked example




```
parent: [0, 0, 0, 0, 4]

rank:   [2, 0, 1, 0, 0]
```

find(3) → 0 with path compression

Java solution

```java
public class UnionFind {
    private final Map<Integer, Integer> parent = new HashMap<>(), rank = new HashMap<>();

    public void createSet(int v) { parent.put(v, v); rank.put(v, 0); }

    public Integer find(int v) {
        if (!parent.containsKey(v)) return null;
        int p = parent.get(v);
        if (p != v) { p = find(p); parent.put(v, p); }
        return p;
    }

    public void union(int a, int b) {
        Integer ra = find(a), rb = find(b);
        if (ra == null || rb == null || ra.equals(rb)) return;
        int ka = rank.get(ra), kb = rank.get(rb);
        if (ka < kb) parent.put(ra, rb);
        else if (ka > kb) parent.put(rb, ra);
        else { parent.put(rb, ra); rank.put(ra, ka + 1); }
    }
}
```

Complexity

- Time: ~O(alpha(n)) amortised per op
- Space: O(n)

Edge cases & pitfalls

- Find on an unknown value → null
- Union of the same set → no-op
- Recursive find depth (iterative alternative)

Interview tips & follow-ups

- Used in Kruskal's MST and connectivity queries
- Follow-up: union by size; count components

Recap

- Each set is a tree with a representative root; path compression and union by rank keep trees flat.
- Complexity: ~O(alpha(n)) amortised per op